Sprint 13: Performance Optimization, Metrics, Tools, Web APIs, and CSS/JavaScript Optimization (අංගසම්පූර්ණ සිංහල අධ්‍යයන සටහන)

0. Where You Are Before Sprint 13 (Sprint 13 ට කලින් ඔයා දැනගත යුතු දේවල්)

- **Concept Explanation (Sinhala):** Sprint 13 පටන් ගන්න කලින්, ඔයාට React වල මූලික සංකල්ප ගැන හොඳ අවබෝධයක් තියෙන්න ඕනේ. අපි ඒ ටික ඉක්මනින් මතක් කරගමු:
  - **React** කියන්නේ user interfaces (UI) හදන්න පාවිච්චි කරන JavaScript library එකක්.
  - **Vite** එකෙන් කරන්නේ React projects ලේසියෙන් create කරලා run කරන්න උදව් වෙන එක.
  - **JSX** කියන්නේ JavaScript ඇතුළේ UI-like syntax ලියන්න දෙන ක්‍රමයක්.
  - **Components** කියන්නේ අපිට ආයෙ ආයෙ පාවිච්චි කරන්න පුළුවන් (reusable) UI කෑලි වලට.
  - **Props** වලින් කරන්නේ parent component එකක ඉඳන් child component එකකට data pass කරන එක. `children` prop එකෙන් එක component එකකට තව JSX කෑල්ලක් wrap කරන්න පුළුවන්.
  - **Conditional rendering** වලින් data වල තත්ත්වය අනුව වෙනස් UI පෙන්වන්න පුළුවන්.
  - Lists render කරන්න අපි `map()` පාවිච්චි කරනවා. React වලට list items ලේසියෙන් track කරන්න **stable keys** දෙන්න ඕනේ.
  - Clicks, typing, සහ form submissions වලට React ප්‍රතිචාර දක්වන්නේ **Events** හරහායි.
  - **Controlled inputs** වලින් form වල values කෙලින්ම React state එකේ store කරනවා.
  - වෙනස් වන data මතක තබා ගැනීමට **useState** hook එක පාවිච්චි කරනවා. මතක තබා ගන්න, state updates හැමවෙලේම **immutable** වෙන්න ඕනේ (කෙලින්ම state එක mutate කරන්න බැහැ).
  - සංකීර්ණ state logic organize කරන්න **useReducer** පාවිච්චි කරනවා.
  - DOM elements වලට access ගන්න හෝ re-render එකක් සිදු නොකර values store කරන්න **useRef** පාවිච්චි කරනවා.
  - React render එකකට පස්සේ side effects run කරන්න **useEffect** පාවිච්චි කරනවා. පැරණි timers, listeners, සහ subscriptions නවත්වන්න **cleanup functions** පාවිච්චි කරනවා.
  - Hook logic නැවත පාවිච්චි කරන්න **custom hooks** හදනවා.
  - App-level එකේ data (user, theme, language වගේ) share කරන්න **Context API** පාවිච්චි කරනවා.
  - APIs හෝ backend සේවාවන් එක්ක සම්බන්ධ වෙන්න **Data fetching** කරන අතර, Loading, data, සහ error states පාවිච්චි කරලා UI එක පැහැදිලිව තබා ගන්නවා.
  - අනවශ්‍ය React වැඩ (calculations, rendering) අඩු කරන්න **useMemo, useCallback,** සහ **React.memo** පාවිච්චි කරනවා.
  - URLs සහ React components සම්බන්ධ කරන්න **React Router** පාවිච්චි කරනවා. හැම page එකක්ම එකපාර load නොවී, අවශ්‍ය වෙලාවට විතරක් load කරන්න **lazy-loaded routes** උදව් වෙනවා.
  - ලොකු apps වල shared state handle කරන්න **State management** tools පාවිච්චි කරනවා.
  - CSS frameworks සහ preprocessors මඟින් styling පිළිවෙළකට තියාගන්නවා.
  - Browser එක HTML, CSS, JavaScript, images, සහ fonts එකතු කරලා screen එකේ pixels බවට පත් කරන ක්‍රියාවලියට **Browser rendering** කියනවා.
  - **Real performance** කියන්නේ අපිට හරියටම මනින්න පුළුවන් speed එකට (measurable speed).
  - **Perceived performance** කියන්නේ user ට app එක කොච්චර වේගවත් කියලා දැනෙනවද කියන එක (user feeling).
  - User කෙනෙක් interaction එකක් කළාට පස්සේ page එක visual විදිහට respond කරන්න කොච්චර වෙලාවක් යනවාද කියලා මනින්නේ **INP** එකෙන්.
  - බලාපොරොත්තු නොවූ විදිහට screen එකේ තියෙන දේවල් එහා මෙහා වෙනකොට **Layout shift** එකක් වෙනවා.

Sprint 12 වලින් ඔයා performance කියන්නේ මොකක්ද කියලා ඉගෙන ගත්තා. Sprint 13 වලින් ඉගෙන ගන්නේ performance මනින්නේ කොහොමද, ප්‍රශ්න හොයාගන්නේ කොහොමද, සහ ඒවා හරියටම විසඳන්නේ (optimize කරන්නේ) කොහොමද කියන එකයි.

1. Sprint 13 Goal (Sprint 13 හි ප්‍රධාන ඉලක්කය)

- **Concept Explanation (Sinhala):** මේ sprint එකේ ප්‍රධානම අරමුණ තමයි React app එකක performance මැනලා (measure), ඒක තවත් වේගවත් කරන එක. මේ පාඩම ඉවර වෙනකොට ඔයාට මේ දේවල් කරන්න පුළුවන් වෙන්න ඕනේ:
  - Optimize කරන්න කලින් measure කරන්න ඕනේ ඇයි කියලා පැහැදිලි කරන්න.
  - Load-time optimization සහ Time to usable කියන්නේ මොකක්ද කියලා තේරුම් ගන්න.
  - UI feedback සහ skeleton screens පාවිච්චි කරලා perceived performance වැඩි කරන්න.
  - FCP, Speed Index, TBT, INP, Bounce Rate, සහ Unique Users වැනි metrics තේරුම් ගන්න.
  - Chrome DevTools Network tab එකෙන් files සහ requests පරීක්ෂා කරන්න.
  - Chrome DevTools Performance tab එකෙන් slow interactions පරීක්ෂා කරන්න.
  - Lighthouse, PageSpeed Insights, සහ WebPageTest පාවිච්චි කරන්න.
  - RUM (Real User Monitoring) tools ගැන ඉගෙන ගන්න.
  - `performance.now()`, Performance Timing API, සහ `PerformanceObserver` පාවිච්චි කරලා code එකක් run වෙන්න යන කාලය මනින්න.
  - Images, caching, compression, සහ CSS optimize කරන්න.
  - React performance tools හරියට පාවිච්චි කරන්න ඉගෙනගෙන පොඩි Performance Dashboard project එකක් කරන්න.

2. The Most Important Sprint 13 Rule (වැදගත්ම රීතිය: අනුමාන කරලා optimize කරන්න එපා!)

- **Concept Explanation (Sinhala):** කිසිම වෙලාවක "App එක slow වගේ දැනෙනවා, ඒ නිසා මම හැමතැනටම useMemo දානවා" කියලා හිතලා වැඩ කරන්න එපා. ඒක වැරදි ක්‍රමයක් (Bad approach).

  Bad Approach (වැරදි ක්‍රමය):

  1. App feels slow -> 2. Randomly add useMemo everywhere -> 3. Randomly lazy-load everything -> 4. Hope the app becomes faster.

  Better Approach (නිවැරදි ක්‍රමය):

  1. Measure first (පළමුව මනින්න) -> 2. Find the bottleneck (වේගය බාල කරන නියම ප්‍රශ්නය හොයන්න) -> 3. Make one improvement (එක් නිශ්චිත වෙනසක් කරන්න) -> 4. Measure again (නැවත මැන බලන්න) -> 5. Keep or reject the change (වෙනස සාර්ථක නම් තබා ගන්න, නැත්නම් ඉවත් කරන්න).

  Optimization කියන්නේ මැනලා තහවුරු කරගත්ත ප්‍රශ්නයක් විසඳලා වඩාත් හොඳ තත්ත්වයට පත් කරන එකටයි. ඔයා measure කරන්නේ නැතුව වැඩ කළොත්, වැරදි දේ optimize කරන්න ඉඩ තියෙනවා (උදාහරණයක් විදිහට ප්‍රශ්නය තියෙන්නේ ලොකු image එකක නම්, ඔයා React code එක optimize කරලා වැඩක් නැහැ).

3. Big Mental Model: Why a React App Can Feel Slow (App එකක් slow වෙන්න බලපාන හේතු)

- **Concept Explanation (Sinhala):** React app එකක් slow වෙන්න හේතු ගොඩක් තියෙන්න පුළුවන්. අපි ප්‍රශ්නයට ගැලපෙනම විසඳුම දෙන්න ඕනේ:
  - **Slow to download?** (Download වෙන්න ගොඩක් වෙලාව යනවා නම්): හේතුව file size එක ලොකු වීම, ලොකු images/fonts තිබීම, හෝ requests වැඩි වීමයි. -> **Fix:** Image compress කරන්න, size එක අඩු කරන්න.
  - **Slow to show first content?** (පළමු content එක screen එකට එන්න වෙලා යනවා නම්): හේතුව render-blocking CSS/JS, නැත්නම් heavy startup work. -> **Fix:** Route lazy-loading හෝ code splitting පාවිච්චි කරන්න.
  - **Slow to become usable?** (UI එක පෙනුනත් click කරන්න/interact කරන්න බැරි නම්): හේතුව interaction එකට කලින් ලොකු JavaScript ප්‍රමාණයක් run වෙන්න තිබීම. -> **Fix:** Non-critical scripts පස්සේ run වෙන්න ඉඩ හරින්න.
  - **Slow after clicking?** (Click කළාට පස්සේ slow නම්): හේතුව heavy event handlers, long tasks, හෝ අනවශ්‍ය rendering. -> **Fix:** Click handler එකේ තියෙන බර වැඩ අඩු කරන්න.
  - **Feels slow even if technically fast?** (Technical පැත්තෙන් fast වුණත් slow වගේ දැනෙනවා නම්): හේතුව loading states, skeleton screens, හෝ UI feedback නැති වීම. -> **Fix:** Loading UI හෝ skeleton screen එකක් දාන්න.

4. Load-Time Optimization

- **Concept Explanation (Sinhala): Load time** කියන්නේ user කෙනෙක් app එක open කරපු වෙලාවේ ඉඳන් එයාට වැඩක් කරගන්න පුළුවන් මට්ටමට content screen එකේ පෙනෙන්න යන කාලයයි. React app එකකට HTML, CSS, JS, Images, Fonts, Icons, සහ API data වැනි files ගොඩක් ඕනේ වෙනවා. මේවා ලොකු වුණොත් load වෙන්න ගොඩක් වෙලාව යනවා.

  පොදු ප්‍රශ්න: ලොකු images/videos, අනවශ්‍ය requests වැඩි වීම, විශාල JS/CSS bundles, caching ක්‍රමවේදයක් නොතිබීම, සහ compression නැති වීම. **මතක තියාගන්න රීතිය:** Browser එකට තවම download කරලා නැති file එකක් run කරන්න හෝ screen එකේ පෙන්වන්න බැහැ!

5. Large Images and Videos (විශාල පින්තූර සහ වීඩියෝ)

- **Concept Explanation (Sinhala):** ලොකු images නිසා මුළු page එකම slow වෙන්න පුළුවන්.

  Bad Image Example: මෙහි `width` සහ `height` නැති නිසා browser එක ඉඩ වෙන් කරන්නේ නැහැ. එම නිසා image එක load වෙද්දී layout එක එහා මෙහා වෙනවා (Layout shift). ඒ වගේම image එක lazy-load වෙන්නෙත් නැහැ.
- **Code Block:**

```
// Bad Image Example
<img src="/hero-large.png" alt="Dashboard preview" />
```

- **Concept Explanation (Sinhala): Better Image Example:** මෙහි WebP format එක පාවිච්චි කරලා තියෙනවා (PNG වලට වඩා size එක අඩුයි). `width` සහ `height` දීලා තියෙන නිසා layout shift වෙන්නේ නැහැ. `loading="lazy"` දැම්ම නිසා screen එකෙන් පිටත තියෙන images අවශ්‍ය වෙනකම් load වෙන්නේ නැහැ.
- **Code Block:**

```
// Better Image Example
<img
  src="/hero.webp"
  width="1200"
  height="600"
  alt="Dashboard preview"
  loading="lazy"
/>
```

- **Concept Explanation (Sinhala): Image Optimization Rules (රීති):**
  1. නිවැරදි size එක පාවිච්චි කරන්න (400px වලින් පෙන්වන තැනකට 4000px image එකක් දාන්න එපා).
  2. WebP හෝ AVIF වැනි modern formats පාවිච්චි කරන්න.
  3. Upload කරන්න කලින් image එක compress කරන්න.
  4. Layout shift වැළැක්වීමට `width` සහ `height` ලබා දෙන්න.
  5. අවශ්‍ය නොවන images වලට `loading="lazy"` පාවිච්චි කරන්න.

6. Too Many Requests (වැඩිපුර requests තිබීම)

- **Concept Explanation (Sinhala):** හැම file එකක්ම download වෙන්න network request එකක් හදනවා. Requests වැඩි වෙනකොට විශේෂයෙන්ම mobile networks වලදී app එක සෑහෙන්න slow වෙනවා.

  විසඳුම: අවශ්‍ය assets විතරක් පාවිච්චි කරන්න, වෙනස් නොවන files cache කරන්න, අනවශ්‍ය scripts/styles අයින් කරන්න. නමුත් හැමදේම එකම giant file එකකට දාන්නත් එපා. මොකද එතකොට user කවදාවත් යන්නේ නැති page එකක කේත පවා පළමු page එකේදීම download කරන්න වෙනවා. හොඳම ක්‍රමය තමයි **Route lazy loading** සහ **Code splitting** පාවිච්චි කරලා අවශ්‍ය වෙලාවට විතරක් අදාළ files download වෙන්න සලස්වන එක.

7. Caching (කැෂින් කිරීම)

- **Concept Explanation (Sinhala): Caching** කියන්නේ browser එක කලින් download කරගත්ත files නැවත පාවිච්චි කරන එකයි. මේ නිසා දෙවැනි පාර app එකට එනකොට අලුතෙන් download වෙන්නේ නැතුව කලින් එකම ඉක්මනට පෙන්නන්න පුළුවන්. Logos, CSS/JS build files, fonts, icons, සහ වෙනස් නොවන images cache කරන්න සුදුසුයි.

  Versioned File Names: Production build එකකදී files වලට `app.9fd23ab.js` වගේ අමුතු hash එකක් එකතු වෙනවා. Code එක වෙනස් වුණොත් මේ hash එක වෙනස් වෙනවා (උදා: `app.41ac902.js`). එතකොට browser එක අලුත් file එකක් කියලා හඳුනාගෙන ඒක විතරක් අලුතෙන් download කරගන්නවා.

8. Minification and Compression

- **Concept Explanation (Sinhala): Minification** කියන්නේ code එකේ තියෙන අනවශ්‍ය හිස්තැන් (spaces), new lines, comments, සහ දිග variable names අයින් කරලා code එකේ size එක කුඩා කරන එකයි.

  Compression කියන්නේ network එක හරහා file එකක් යවන්න කලින් සේවාදායකයේදී (server) gzip හෝ Brotli වැනි තාක්ෂණයක් පාවිච්චි කරලා file එක තවත් කුඩා කරලා යවන එකයි. Browser එකට ලැබුණාම ඒක decompress කරලා පාවිච්චි කරනවා.

  - සාරාංශය: Minification මඟින් code එක කුඩා කරන අතර, Compression මඟින් එය වේගයෙන් network එක හරහා ප්‍රවාහනය කරයි.

9. Time to Usable (භාවිතයට ගත හැකි වන තෙක් ගතවන කාලය)

- **Concept Explanation (Sinhala):** සමහර වෙලාවට app එක screen එකේ පෙනුනත් user ට click කරන්න බැහැ, මොකද JavaScript එක තවම background එකේ බර වැඩක් කරමින් ඉන්න නිසා (app freeze වෙලා වගේ දැනෙනවා).

  Time to usable කියන්නේ user ට කිසිම බාධාවක් නැතුව app එක සමඟ interact කරන්න (click, type, scroll) පුළුවන් මට්ටමට පත්වෙන්න යන කාලයටයි.

10. Improving Time to Usable (Time to Usable වැඩි දියුණු කිරීම)

- **Concept Explanation (Sinhala): * Show Critical Content First:** වැදගත් content එක මුලින්ම පෙන්වන්න. බර දේවල් (charts, maps, animations, widgets) පස්සේ load වෙන්න සලස්වන්න.
  - **Defer Non-Critical Scripts:** Analytics වගේ පළමු screen එකට අදාළ නැති scripts වලට `defer` attribute එක පාවිච්චි කරන්න. එවිට HTML parse වෙන එක බ්ලොක් වෙන්නේ නැහැ.
- **Code Block:**

```
<!-- Better approach with defer -->
<h1>Welcome</h1>
<script src="analytics.js" defer></script>
```

- **Concept Explanation (Sinhala):**
  - **Lazy-Load Large Route Pages:** ලොකු pages (dashboard, admin panel වගේ ඒවා) මුලින්ම load කරන්නේ නැතුව, user ඒ page එකට යනකොට විතරක් load කරන්න React වල `lazy` සහ `Suspense` පාවිච්චි කරන්න පුළුවන්. හැබැයි පොඩි buttons, labels වගේ හැම කුඩා component එකක්ටම `lazy` පාවිච්චි කරන්න එපා, ඒකෙන් code එක අනවශ්‍ය ලෙස සංකීර්ණ වෙනවා.
- **Code Block:**

```
import { lazy, Suspense } from "react";

const Dashboard = lazy(() => import("./pages/Dashboard"));

export default function App() {
  return (
    <Suspense fallback={<p>Loading dashboard...</p>}>
      <Dashboard />
    </Suspense>
  );
}
```

11. Perceived Performance (පාරිභෝගිකයාට දැනෙන වේගය)

- **Concept Explanation (Sinhala):** App එකක ඇත්තටම වැඩක් වෙන්න තත්පර 2ක් ගියත්, user ට ඒක ක්ෂණිකව දැනෙන්න සලස්වන්න පුළුවන් (UI feedback දීමෙන්). User button එකක් click කළාට පස්සේ කිසිම දෙයක් සිදු නොවුනොත්, එයා හිතන්නේ app එක කැඩිලා කියලා නැවත නැවත click කරන්න ඉඩ තියෙනවා. ඒ නිසා හැමවෙලේම button එක click කළ සැනින් "Saving..." හෝ "Loading..." වගේ feedback එකක් සහ button එක `disabled` කිරීමක් කරන්න ඕනේ.

12. Loading UI and Skeleton Screens

- **Concept Explanation (Sinhala):** සරල වැඩකට නම් සරල `Loading...` text එකක් හොඳයි. නමුත් ලොකු dashboard එකක් වගේ page එකකට **Skeleton Screen** එකක් (ඇත්තම content එකේ හැඩය ගත් placeholder layout එකක්) පෙන්වන එක වඩාත් වෘත්තීය මට්ටමේ වේගවත් හැඟීමක් ලබා දෙනවා.
- **Code Block:**

```
// Skeleton UI inside Component
if (loading) {
  return (
    <main>
      <div className="skeleton title"></div>
      <div className="skeleton line"></div>
      <div className="skeleton line"></div>
    </main>
  );
}
```

```
/* Skeleton CSS definition */
.skeleton {
  background: #e5e7eb;
  border-radius: 8px;
  margin-bottom: 12px;
}
.skeleton.title {
  width: 50%;
  height: 32px;
}
.skeleton.line {
  width: 100%;
  height: 16px;
}
```

13. Performance Metrics (මිනුම් දඬු)

- **Concept Explanation (Sinhala):** අපිට app එකක වේගය මනින්න විවිධ metrics පාවිච්චි කරන්න පුළුවන්. අපි ඒවා එකින් එක බලමු:

14. FCP: First Contentful Paint

- **Concept Explanation (Sinhala): FCP** එකෙන් මනින්නේ user page එකට ආවට පස්සේ පළමු පින්තූරය, අකුර, SVG එක හෝ මොකක් හරි content එකක් screen එකේ දකින්න යන කාලයයි. FCP එක slow වුණොත් user ට ගොඩක් වෙලාව හිස් screen එකක් (blank screen) බලාගෙන ඉන්න වෙනවා.

15. Speed Index

- **Concept Explanation (Sinhala): Speed Index** එකෙන් මනින්නේ screen එකේ පෙනෙන මුළු කොටසම visual විදිහට සම්පූර්ණයෙන්ම පිරී ඉතිරී යන්න (complete වෙන්න) යන කාලයයි.
  - වෙනස: FCP එකෙන් කියන්නේ "කවදාද පළමු දේ පෙනුනේ?" කියන එකයි. Speed Index එකෙන් කියන්නේ "කොච්චර ඉක්මනට මුළු page එකම පිරිලා පෙනුනද?" කියන එකයි.

16. TBT: Total Blocking Time

- **Concept Explanation (Sinhala): TBT** එකෙන් මනින්නේ page එක load වෙන අතරතුර JavaScript වැඩ නිසා browser එකේ **main thread** එක බ්ලොක් වෙලා තිබුණු මුළු කාලයයි. Main thread එක බ්ලොක් වුණාම user click කළත් page එක respond කරන්නේ නැහැ.

17. INP: Interaction to Next Paint

- **Concept Explanation (Sinhala): INP** එකෙන් මනින්නේ user කෙනෙක් click එකක් හෝ key press එකක් කළාට පස්සේ, browser එක ඊළඟ visual update එක (frame එක) screen එකේ paint කරන්න ගන්න කාලයයි. INP අගය අඩු වෙන තරමට app එකේ responsiveness එක වැඩියි.

18. Bounce Rate (පිටවීමේ ප්‍රතිශතය)

- **Concept Explanation (Sinhala): Bounce Rate** කියන්නේ user කෙනෙක් app එකේ එක page එකක් විතරක් බලලා, වෙන කිසිම තැනක් click කරන්නේ නැතුව app එකෙන් පිටවෙලා යන ප්‍රතිශතයයි. App එක slow වීම, නරක design එක, හෝ navigation අපැහැදිලි වීම නිසා bounce rate එක වැඩි වෙන්න පුළුවන්.

19. Unique Users (අද්විතීය පරිශීලකයින්)

- **Concept Explanation (Sinhala): Unique Users** කියන්නේ නිශ්චිත කාලයක් ඇතුළත app එකට ආපු වෙනස්ම පුද්ගලයින් ගණනයි. අපිට performance ප්‍රශ්න විසඳීමේදී ප්‍රමුඛතාවය දෙන්න මේක උදව් වෙනවා. (උදා: පරිශීලකයින් 5 දෙනෙකුට බලපාන ප්‍රශ්නයකට වඩා පරිශීලකයින් 30,000 කට බලපාන ප්‍රශ්නයක් අපි ඉක්මනින් විසඳන්න ඕනේ).

20. Tools for Performance Testing (Performance මනින Tools)

- **Concept Explanation (Sinhala):** අනුමාන කරන්නේ නැතුව හරියටම ප්‍රශ්නය තියෙන්නේ කොහේද කියලා බලන්න අපිට විවිධ tools පාවිච්චි කරන්න පුළුවන්.

21. Chrome DevTools: Network Tab

- **Concept Explanation (Sinhala):** Network tab එකෙන් අපේ page එකට load වෙන හැම file එකක්ම, ඒවායේ size එක, load වෙන්න ගිය වෙලාව, cache සහ compression තත්ත්වය බලාගන්න පුළුවන්.

  Beginner Exercise:

  1. App එක run කරලා Chrome DevTools වල Network tab එකට යන්න.
  2. Page එක reload කරන්න.
  3. Size column එකෙන් sort කරලා ලොකුම file එක මොකක්ද කියලා බලන්න.
  4. ඒ file එක initial load එකටම ඕනෙද කියලා හිතලා බලන්න.

22. Chrome DevTools: Performance Tab

- **Concept Explanation (Sinhala):** Performance tab එකෙන් අපිට page එක record කරලා, load වෙද්දී හෝ click එකක් කරද්දී browser එක ඇතුළේ වෙන දේවල් (script running, rendering, layout, painting) සවිස්තරාත්මකව බලාගන්න පුළුවන්. මෙහිදී **Long Task** (main thread එක මඟින් 50ms වඩා ගන්නා වැඩ) අපිට රතු පාටින් බලාගන්න පුළුවන්.

23. Lighthouse

- **Concept Explanation (Sinhala): Lighthouse** කියන්නේ browser එකෙන්ම අපේ site එකේ Performance, Accessibility, Best Practices, සහ SEO ගැන වාර්තාවක් (Report) දෙන tool එකක්.
  - වැදගත්: 100/100 score එකක් ගන්න අන්ධයෙක් වගේ හැම තැනම වෙනස්කම් කරන්න එපා. වාර්තාව කියවලා ඇත්තටම user ට වැදගත් වෙන ප්‍රශ්න විතරක් විසඳන්න.

24. PageSpeed Insights and WebPageTest

- **Concept Explanation (Sinhala): * PageSpeed Insights:** App එක deploy කළාට පස්සේ online link එක දීලා performance මනින්න පාවිච්චි කරන Google tool එකක්.
  - **WebPageTest:** විවිධ රටවල් වල ඉඳන්, විවිධ network speeds (3G, 4G වගේ) යටතේ app එක load වෙන්නේ කොහොමද කියලා ගැඹුරින් විශ්ලේෂණය කරන්න පාවිච්චි කරන tool එකක්.

25. RUM Tools (Real User Monitoring)

- **Concept Explanation (Sinhala):** අපේ පරිගණකය වේගවත් වුණාට real users ලා පාවිච්චි කරන්නේ slow phones, දුර්වල 3G networks වෙන්න පුළුවන්. **RUM** tools වලින් කරන්නේ ඇත්තම users ලාගේ devices වලින් ලැබෙන performance metrics මැනලා අපිට පෙන්වන එකයි (Lab tests වලින් වෙන්නේ පාලනය කළ පරිසරයක මනින එකයි).

26. Web Performance APIs

- **Concept Explanation (Sinhala):** අපිට browser එක හරහාම JavaScript කේත පාවිච්චි කරලා performance මනින්න පුළුවන් APIs වර්ග කිහිපයක් තියෙනවා:

27. performance.now()

- **Concept Explanation (Sinhala):** `performance.now()` මඟින් ඉතාමත් නිවැරදි කාල සලකුණක් (high-precision timestamp) මිලි තත්පර වලින් ලබා දෙනවා. යම් code block එකක් run වෙන්න කොච්චර වෙලාවක් ගියාද කියලා බලන්න මේක පාවිච්චි කරන්න පුළුවන්.
- **Code Block:**

```
const start = performance.now();

let total = 0;
for (let i = 0; i < 10000000; i++) {
  total += i;
}

const end = performance.now();
console.log(`Work took ${end - start}ms`);
```

28. Performance Timing API

- **Concept Explanation (Sinhala):** මුළු page එකම load වෙන්න කොච්චර වෙලාවක් ගියාද වගේ විස්තර browser එකෙන් කෙලින්ම ලබා ගන්න මේ API එක පාවිච්චි කරනවා.
- **Code Block:**

```
window.addEventListener("load", () => {
  const timing = performance.timing;
  const pageLoadTime = timing.loadEventEnd - timing.navigationStart;
  console.log("Page load time:", pageLoadTime, "ms");
});
```

29. PerformanceObserver

- **Concept Explanation (Sinhala):** `PerformanceObserver` එකෙන් කරන්නේ app එක run වන අතරතුර සිදුවන විවිධ performance මිනුම් (measures) ක්‍රමවත් ආකාරයකට බලාගෙන සිටීම (observe කිරීම) යි. අපිට `performance.mark()` දාලා පටන් ගන්න තැන සහ ඉවර වෙන තැන ලකුණු කරලා, `performance.measure()` එකෙන් ඒ අතර කාලය මනින්න පුළුවන්.
- **Code Block:**

```
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log(entry.name, entry.duration);
  }
});
observer.observe({ entryTypes: ["measure"] });

performance.mark("start-work");

let total = 0;
for (let i = 0; i < 5000000; i++) {
  total += i;
}

performance.mark("end-work");
performance.measure("heavy-work", "start-work", "end-work");
```

30. CSS Performance

- **Concept Explanation (Sinhala):** CSS ලිවීමත් performance වලට බලපානවා. අනවශ්‍ය CSS, ඉතා ගැඹුරු selectors, සහ layout-heavy animations නිසා browser එකට style ගලපන්න සහ screen එකේ අඳින්න (paint කරන්න) වැඩි වෙලාවක් යනවා.

31. Remove Unused CSS (පාවිච්චි නොකරන CSS ඉවත් කිරීම)

- **Concept Explanation (Sinhala):** අපේ app එකේ තියෙන්නේ සරල buttons 2කුයි card එකකුයි විතරක් නම්, මුළු CSS framework එකකම තියෙන දහස් ගණන් CSS රීති import කරගන්න එක වැරදියි. අවශ්‍ය දේ විතරක් පාවිච්චි කරන්න.

32. Avoid Very Deep CSS Selectors

- **Concept Explanation (Sinhala):** ඉතා දිගු selectors ලිවීමෙන් browser එකට ඒවා ගලපන්න වැඩි වෙලාවක් යනවා වගේම code එක කියවන්න සහ නඩත්තු කරන්න අමාරු වෙනවා.

  Bad Selector Example:
- **Code Block:**

```
/* Bad Deep Selector */
.app.page.content.section .card .header .title span {
  color: red;
}
```

- **Concept Explanation (Sinhala): Better Selector Example:** සරලව class එකක් ලියලා ඒකට style දීම වඩාත් වේගවත් සහ පිරිසිදු ක්‍රමයයි.
- **Code Block:**

```
/* Better Flat Selector */
.card-title {
  color: red;
}
```

33. Avoid Layout-Heavy Animations (Layout එක වෙනස් කරන animations වලක්වන්න)

- **Concept Explanation (Sinhala):** `width`, `height`, `top`, `left`, `margin` වැනි දේවල් transition හෝ animate කරන විට browser එකට නැවත නැවතත් layout එක ගණනය කරන්න වෙනවා (expensive).

  Bad Animation Example:
- **Code Block:**

```
/* Bad transition that triggers layout */
.box {
  transition: width 300ms;
}
.box:hover {
  width: 500px;
}
```

- **Concept Explanation (Sinhala): Better Animation Example:** `transform` සහ `opacity` animate කිරීම ඉතා ලාභදායී (cheap) ක්‍රමයක්, මොකද ඒවා කෙලින්ම GPU එකෙන් handle කරන්න පුළුවන්.
- **Code Block:**

```
/* Better transition using transform */
.box {
  transition: transform 300ms;
}
.box:hover {
  transform: scale(1.05);
}
```

34. Limit Font Variants (Font variants සීමා කරන්න)

- **Concept Explanation (Sinhala):** Roboto font එකේම regular, medium, bold, italic, black වගේ හැම variant එකක්ම load කරන්න ගියොත් download size එක ගොඩක් වැඩි වෙනවා. App එකට ඇත්තටම ඕනෙ වෙන weights (උදා: Regular සහ Bold) විතරක් තෝරලා පාවිච්චි කරන්න.

35. JavaScript Performance (JavaScript ක්‍රියාකාරීත්වය ප්‍රශස්ත කිරීම)

- **Concept Explanation (Sinhala):** JavaScript run වෙන්නේ browser එකේ main thread එකේ නිසා බර වැඩක් (heavy loops, expensive calculations) කෙලින්ම run කළොත් UI එක freeze වෙනවා.

36. Heavy Click Handler Problem (බරැති Click Handler ප්‍රශ්නය)

- **Concept Explanation (Sinhala): Bad Example:** මෙහි button එක click කළාම loop එකක් ඇතුළේ බිලියන 2ක් වෙනකම් ගණන් කරනවා. මේ වෙලාවේ browser එක සම්පූර්ණයෙන්ම freeze වෙනවා.
- **Code Block:**

```
// Bad Heavy Calculation on Main Thread
function SearchButton() {
  function handleSearch() {
    let total = 0;
    for (let i = 0; i < 2000000000; i++) {
      total += i;
    }
    console.log(total);
  }
  return <button onClick={handleSearch}>Search</button>;
}
```

- **Concept Explanation (Sinhala): Better Beginner Version:** මෙහිදී අපි මුලින්ම state එක "Searching..." කරලා UI එකට feedback එක දෙනවා. ඊට පස්සේ `setTimeout(..., 0)` දාලා බර වැඩේ පටන් ගන්න එක පොඩ්ඩක් පස්සට දානවා. එවිට browser එකට UI එක update කරලා ඉවර වෙලා ඒ බර වැඩේ කරන්න පුළුවන්. (සැබෑ ලෝකයේ මීට වඩා ලොකු calculations වලට Web Workers පාවිච්චි කරනවා).
- **Code Block:**

```
import { useState } from "react";

export default function SearchButton() {
  const [status, setStatus] = useState("Idle");

  function handleSearch() {
    setStatus("Searching...");

    setTimeout(() => {
      let total = 0;
      for (let i = 0; i < 50000000; i++) {
        total += i;
      }
      console.log(total);
      setStatus("Done");
    }, 0);
  }

  return (
    <section>
      <button type="button" onClick={handleSearch}>
        Search
      </button>
      <p>{status}</p>
    </section>
  );
}
```

37. React-Specific Performance Reminders (React වලටම ආවේණික මතක් කිරීම්)

- **Concept Explanation (Sinhala):** `useMemo`, `useCallback`, සහ `React.memo` කියන්නේ magic solutions නෙමෙයි. ඒවා හැම තැනම දාන්න කලින් මැනලා බලන්න ඕනේ.

38. useMemo Reminder

- **Concept Explanation (Sinhala): Good Use of useMemo:** මෙහි `products` array එක සෑහෙන්න ලොකු එකක් නම්, query එක වෙනස් නොවුණත් වෙනත් කිසිම state එකක් නිසා re-render එකක් වුණොත්, මුළු array එකම ආයෙ ආයෙ filter වෙන එක නවත්වන්න `useMemo` පාවිච්චි කරන එක සුදුසුයි.
- **Code Block:**

```
import { useMemo, useState } from "react";

export default function ProductSearch({ products }) {
  const [query, setQuery] = useState("");

  const filteredProducts = useMemo(() => {
    return products.filter((product) =>
      product.name.toLowerCase().includes(query.toLowerCase())
    );
  }, [products, query]);

  return (
    <main>
      <input
        value={query}
        onChange={(event) => setQuery(event.target.value)}
        placeholder="Search products"
      />
      <ul>
        {filteredProducts.map((product) => (
          <li key={product.id}>{product.name}</li>
        ))}
      </ul>
    </main>
  );
}
```

- **Concept Explanation (Sinhala): Bad Use of useMemo:** නමුදු සරල string එකක් එකතු කරන්න `useMemo` දැමීම අනවශ්‍යයි, මොකද සරල ගණනය කිරීම් වලට වඩා `useMemo` එකට යන overhead එක වැඩි වෙන්න පුළුවන්.
- **Code Block:**

```
// Unnecessary useMemo
const fullName = useMemo(() => firstName + " " + lastName, [firstName,
lastName]);

// Better Approach
const fullName = firstName + " " + lastName;
```

39. useCallback Reminder

- **Concept Explanation (Sinhala):** `useCallback` පාවිච්චි කරන්නේ component එකක් re-render වෙද්දී function reference එක අලුතෙන් හැදෙන එක නවත්වන්නයි. මෙය වැදගත් වෙන්නේ එම function එක prop එකක් විදිහට `React.memo` කරපු child component එකකට යවන කොට විතරයි. හැම function එකකටම නිකරුණේ දාන්න එපා.

40. React.memo Reminder

- **Concept Explanation (Sinhala):** `React.memo` වලින් component එකකට ලැබෙන props වෙනස් නොවුණොත් re-render වෙන එක වළක්වනවා. මෙය පාවිච්චි කළ යුත්තේ re-render වෙන්න ලොකු වෙලාවක් යන (heavy render) child component එකක props නිතරම සමානව පවතින අවස්ථා වලදී විතරයි.

41. Rendering Huge Lists (විශාල lists render කිරීම්)

- **Concept Explanation (Sinhala):** Array එකක items 20,000ක් තිබ්බොත්, ඒ 20,000ම එකපාර screen එකේ අඳින්න ගියොත් browser එක හිර වෙනවා.

  විසඳුම: සරලව `.slice(0, 20)` වගේ දාලා මුලින්ම පෙනෙන සීමිත ප්‍රමාණයක් (උදා: items 20ක්) විතරක් render කරලා පෙන්වන්න. (සංකීර්ණ apps වලදී මේ සඳහා List Virtualization පාවිච්චි කරනවා).

42 - 45. Sprint 13 Mini Project: Performance Dashboard (කුඩා ව්‍යාපෘතිය)

- **Concept Explanation (Sinhala):** මෙම ව්‍යාපෘතියෙන් `useMemo` පාවිච්චි කරලා ලොකු list එකක් filter කරන හැටි, `performance.now()` දාලා කාලය මනින හැටි, Loading status පෙන්වන හැටි, සහ image optimization කරන හැටි ප්‍රායෝගිකව ඉගෙන ගන්න පුළුවන්.
- **Code Block:**

```
// src/App.jsx
import { useMemo, useState } from "react";
import "./App.css";

const products = Array.from({ length: 5000 }, (_, index) => ({
  id: index + 1,
  name: `Product ${index + 1}`,
  price: Math.floor(Math.random() * 100) + 1,
}));

export default function App() {
  const [query, setQuery] = useState("");
  const [status, setStatus] = useState("No measurement yet.");
  const [duration, setDuration] = useState(null);

  const filteredProducts = useMemo(() => {
    return products.filter((product) =>
      product.name.toLowerCase().includes(query.toLowerCase())
    );
  }, [query]);

  function handleMeasureWork() {
    setStatus("Measuring heavy work...");
    setTimeout(() => {
      const start = performance.now();
      let total = 0;
      for (let i = 0; i < 50000000; i++) {
        total += i;
      }
      const end = performance.now();
      setDuration(end - start);
      setStatus("Measurement complete.");
    }, 0);
  }

  return (
    <main className="page">
      <section className="hero">
        <h1>Performance Dashboard</h1>
        <p>
          Sprint 13 teaches measurement first, then optimization. Do not
guess.
        </p>
      </section>

      <section className="card">
        <h2>Search Products</h2>
        <label htmlFor="search">Search</label>
        <input
          id="search"
          value={query}
          onChange={(event) => setQuery(event.target.value)}
          placeholder="Try Product 100"
        />
        <p>Showing {filteredProducts.length} products.</p>
        <ul>
          {filteredProducts.slice(0, 20).map((product) => (
            <li key={product.id}>
              {product.name} ${product.price}
            </li>
          ))}
        </ul>
      </section>

      <section className="card">
        <h2>Measure Heavy Work</h2>
        <button type="button" onClick={handleMeasureWork}>
          Measure work
        </button>
        <p>{status}</p>
        {duration !== null && (
          <p>Heavy work took {duration.toFixed(2)}ms.</p>
        )}
      </section>

      <section className="card">
        <h2>Optimized Image Example</h2>
        <img
          src="/performance-preview.webp"
          width="800"
          height="450"
          alt="Performance dashboard preview"
          loading="lazy"
        />
      </section>
    </main>
  );
}
```

```
/* src/App.css */
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f8fafc;
  color: #111827;
}

.page {
  width: min(900px, 100%);
  margin: 0 auto;
  padding: 24px;
}

.hero,
.card {
  background: white;
  border: 1px solid #e5e7eb;
  border-radius: 16px;
  padding: 20px;
  margin-bottom: 20px;
}

input {
  display: block;
  width: 100%;
  padding: 10px;
  margin: 8px 0 12px;
}

button {
  padding: 10px 14px;
  cursor: pointer;
}

img {
  max-width: 100%;
  height: auto;
  display: block;
}
```

46 - 49. Debug Task & Better Solutions (වැරදි කේත නිවැරදි කිරීම)

- **Concept Explanation (Sinhala): Bad HTML/JS Code:** මෙහි තියෙන වැරදි:
  1. `large-analytics.js` script එක මුලින්ම තියෙන නිසා HTML parsing block වෙනවා.
  2. Image එකේ format එක (PNG) ලොකු වැඩියි වගේම `width`, `height` සහ `loading="lazy"` නැහැ.
  3. `onclick` inline handler එක React වලට ගැලපෙන්නේ නැහැ.
- **Code Block:**

```
<!-- Bad Code Snippet -->
<script src="large-analytics.js"></script>
<img src="hero.png" alt="Hero image" />
<button onclick="runVeryHeavyWork()">Search</button>
```

- **Concept Explanation (Sinhala): Better HTML/JS Version:** මෙහි වැදගත් content එක කලින් පෙනෙන අතර, script එකට `defer` දමා ඇත. Image එක WebP කර, width/height සහ lazy loading දමා ඇත.
- **Code Block:**

```
<!-- Better HTML Setup -->
<h1>Search Page</h1>
<img
  src="hero.webp"
  width="1200"
  height="600"
  alt="Hero image"
  loading="lazy"
/>
<script src="large-analytics.js" defer></script>
```

- **Concept Explanation (Sinhala): Better React Button Version:** කෙලින්ම inline JS නැතිව React style එකෙන් `onClick` event handler එකක් ලියා, status feedback සහ `setTimeout` මඟින් UI update වීමට ඉඩ සලසා ඇත.
- **Code Block:**

```
// Better React Component Design
import { useState } from "react";

function SearchAction() {
  const [status, setStatus] = useState("Ready");

  function handleSearch() {
    setStatus("Searching...");
    setTimeout(() => {
      // Do reduced or delayed work here.
      setStatus("Done");
    }, 0);
  }

  return (
    <>
      <button type="button" onClick={handleSearch}>
        Search
      </button>
      <p>{status}</p>
    </>
  );
}
```

50. Common Sprint 13 Beginner Mistakes (ආධුනිකයින් කරන පොදු වැරදි)

- **Concept Explanation (Sinhala): * Mistake 1: Optimizing Without Measuring:** මනින්නේ නැතුව අනුමාන කරලා useMemo හැමතැනම දාන්න එපා. DevTools වලින් ප්‍රශ්නය හරියටම සොයන්න.
  - **Mistake 2: Thinking Performance Is Only React:** React code එක විතරක් නෙමෙයි performance කියන්නේ. Network, Images, CSS, Caching, Fonts, සහ Compression යන හැමදේම බලපානවා.
  - **Mistake 3: Lazy-Loading Everything:** හැම පොඩි button එකක්ම lazy-load කරන්න එපා. ලොකු pages, rare modals, හෝ බර routes විතරක් lazy-load කරන්න.
  - **Mistake 4: Ignoring Perceived Performance:** වැඩක් සිද්ධ වෙද්දී user ට මුකුත්ම පෙන්වන්නෙ නැතුව ඉන්න එපා. Loading text, skeletons, හෝ disabled buttons පාවිච්චි කරන්න.
  - **Mistake 5: Rendering Too Much at Once:** එකපාර items 10,000ක් render කරන්න එපා. `.slice` කරලා මුල් 20ක් විතරක් පෙන්වන්න.
  - **Mistake 6: Using Huge Images:** 5MB image එකක් thumbnail එකක් විදිහට පාවිච්චි කරන්න එපා. Compress කරලා, size එක අඩු කරන්න.
  - **Mistake 7: Animating Layout Properties:** `width` transition කරනවා වෙනුවට `transform: scale()` පාවිච්චි කරන්න.

51. Revision Checkpoint Questions (ස්වයං ඇගයීම් ප්‍රශ්න)

- **Concept Explanation (Sinhala):** පාඩම අවසානයේ ඔයාට මේ ප්‍රශ්න වලට තනිවම උත්තර දෙන්න පුළුවන්ද බලන්න:
  1. Optimize කරන්න කලින් measure කරන්න ඕනේ ඇයි?
  2. Time to usable කියන්නේ මොකක්ද?
  3. FCP සහ Speed Index අතර වෙනස මොකක්ද?
  4. TBT සහ INP මඟින් මනින්නේ මොනවාද?
  5. Caching වලදී versioned file names (hashes) උදව් වෙන්නේ කොහොමද?
  6. Animations වලදී `transform` වඩාත් හොඳ ඇයි?
  7. `performance.now()` පාවිච්චි කරන්නේ ඇයි?
  8. React.memo පාවිච්චි කරන්න සුදුසුම අවස්ථාව මොකක්ද?

52. Shortest Correct Mental Model (කෙටිම සහ නිවැරදිම මානසික ආකෘතිය)

- **Concept Explanation (Sinhala):** React app එකක් නිකම්ම වැඩ කළාට මදි. එය මැනිය යුතුය (Measured), ප්‍රශස්ත කළ යුතුය (Optimized), සහ නැවත තහවුරු කළ යුතුය (Verified).

  නිවැරදි පියවර: `Measure first -> Find the real bottleneck -> Fix the bottleneck -> Measure again`

  අවසාන රන් රීතිය: කිසිම විටක අනුමාන කරන්න එපා! පළමුව මනින්න, ප්‍රවේශමෙන් වැඩි දියුණු කරන්න, ඉන්පසු ප්‍රතිඵලය තහවුරු කරගන්න (Do not guess. Measure first, improve carefully, then verify the result).